pandas

The Complete Pandas Guide

Data Structures, Operations, and Worked Examples for Learning and Reference (Python Library Series)

Covers: Series & DataFrame • I/O • Indexing & Filtering • Missing Data • GroupBy & Aggregation • Merge/Join/Concat • Pivot Tables • Apply/Map • Strings & Dates • Cheat Sheet

import pandas as pd | Reference version: pandas 2.x | Format: concept explanation + runnable example + expected output

1. Introduction to Pandas

Pandas is an open-source Python library for fast, flexible, and expressive data manipulation and analysis. It is built on top of NumPy and is the standard tool in Python for working with structured (tabular) data: spreadsheets, CSV files, SQL query results, and time series.

- Provides two core data structures: **Series** (1-dimensional) and **DataFrame** (2-dimensional).
- Handles missing data, alignment, reshaping, merging, and grouping out of the box.
- Reads and writes many formats: CSV, Excel, JSON, SQL, Parquet, HTML, and more.
- Integrates with NumPy, Matplotlib, scikit-learn, and most of the Python data science stack.

Installation and Import

Install and import pandas

```
pip install pandas

import pandas as pd
import numpy as np

print(pd.__version__)
```

Output:

```
2.2.2
```

Note: The alias 'pd' is a near-universal convention; almost all pandas code and documentation assumes it.

2. Core Data Structures

2.1 Series — a labeled 1D array

A **Series** is a one-dimensional labeled array capable of holding any data type (integers, strings, floats, Python objects). The labels are collectively called the **index**.

Creating a Series

```
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
print(s)
print(s['b'])       # access by label
print(s.values)     # underlying NumPy array
print(s.index)      # the index object
```

Output:

```
a    10
b    20
c    30
d    40
dtype: int64
20
[10 20 30 40]
Index(['a', 'b', 'c', 'd'], dtype='object')
```

2.2 DataFrame — a labeled 2D table

A **DataFrame** is a two-dimensional table of rows and columns, similar to a spreadsheet or SQL table. Each column is effectively a Series, and all columns share the same row index.

Creating a DataFrame from a dictionary

```
data = {
    'name': ['Nour', 'Ali', 'Sara', 'Omar'],
    'age': [22, 25, 21, 23],
    'city': ['Alexandria', 'Cairo', 'Giza', 'Aswan']
}
df = pd.DataFrame(data)
print(df)
```

Output:

```
    name  age       city
0   Nour   22  Alexandria
1    Ali   25      Cairo
2   Sara   21       Giza
3   Omar   23      Aswan
```

Creating a DataFrame from a list of lists / NumPy array

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
df2 = pd.DataFrame(arr, columns=['a', 'b', 'c'])
print(df2)
```

Output:

```
   a  b  c
0  1  2  3
1  4  5  6
```

3. Reading and Writing Data

Pandas provides a consistent pd.read_* / DataFrame.to_* API across many file formats.

Function	Purpose
`pd.read_csv('f.csv')`	Read a CSV file into a DataFrame
`pd.read_excel('f.xlsx')`	Read an Excel sheet
`pd.read_json('f.json')`	Read a JSON file
`pd.read_sql(query, conn)`	Read the result of a SQL query
`pd.read_parquet('f.parquet')`	Read a Parquet file
`df.to_csv('out.csv', index=False)`	Write a DataFrame to CSV
`df.to_excel('out.xlsx', index=False)`	Write a DataFrame to Excel
`df.to_json('out.json')`	Write a DataFrame to JSON

Reading and writing CSV files

```
df = pd.read_csv('students.csv')
df.to_csv('students_copy.csv', index=False)  # index=False avoids saving row numbers
```

Note: Common read_csv arguments: sep=',', header=0, names=[...], usecols=[...], dtype={...}, parse_dates=['date_col'], na_values=['NA', '?'].

4. Inspecting Data

Quick overview of a DataFrame

```
df.head()        # first 5 rows
df.tail(3)       # last 3 rows
df.shape         # (rows, columns)
df.columns       # column names
df.dtypes        # data type of each column
df.info()        # summary: dtypes, non-null counts, memory usage
df.describe()    # summary statistics for numeric columns
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 3 columns):
 # Column  Non-Null Count  Dtype
--- ------  --------------  -----
 0 name    4 non-null      object
 1 age     4 non-null      int64
 2 city    4 non-null      object
```

5. Selecting and Indexing Data

5.1 Selecting columns

Column selection

```
df['name']              # single column -> Series
df[['name', 'age']]     # multiple columns -> DataFrame
```

5.2 loc vs iloc

.loc selects by *label* (row/column names). **.iloc** selects by *integer position*. Both take the form [rows, columns].

Row and cell selection with loc / iloc

```
df.loc[0]                # row with label/index 0
df.loc[0, 'name']        # single cell by label
df.loc[0:2, ['name','age']]  # label-based slice (inclusive of 2)

df.iloc[0]               # first row by position
df.iloc[0, 1]            # cell at row 0, col 1
df.iloc[0:2, 0:2]        # position-based slice (exclusive of end)
```

Note: A frequent gotcha: .loc slices are INCLUSIVE of the end label, while .iloc slices are EXCLUSIVE of the end position, matching normal Python slicing.

6. Filtering with Boolean Indexing

Filtering rows with conditions

```
df[df['age'] > 22]

# multiple conditions need parentheses and & / | (not 'and'/'or')
df[(df['age'] > 21) & (df['city'] == 'Cairo')]

df[df['city'].isin(['Cairo', 'Giza'])]
df[~df['city'].isin(['Cairo'])]     # ~ negates the condition
```

7. Handling Missing Data

Detecting and handling NaN values

```
df.isna()             # boolean mask of missing values
df.isna().sum()       # count of missing values per column

df.dropna()                     # drop rows with any NaN
df.dropna(subset=['age'])       # drop rows where 'age' is NaN

df.fillna(0)                    # replace NaN with 0
df['age'].fillna(df['age'].mean(), inplace=True)  # fill with column mean
```

Note: inplace=True modifies the DataFrame directly and returns None; most pandas code prefers reassignment (df = df.fillna(...)) since inplace is being phased toward deprecation in several methods.

8. Adding, Modifying, and Dropping Columns

Column operations

```
df['age_plus_5'] = df['age'] + 5          # new column from arithmetic
df['is_adult'] = df['age'] >= 18           # new boolean column
df.rename(columns={'name': 'full_name'}, inplace=True)
df.drop(columns=['age_plus_5'], inplace=True)
df.drop(index=0)                            # drop row with label 0
```

9. Sorting

Sorting by values and by index

```
df.sort_values('age')                       # ascending
df.sort_values('age', ascending=False)       # descending
df.sort_values(['city', 'age'])              # sort by multiple columns
df.sort_index()                              # sort by row index
```

10. GroupBy and Aggregation

groupby follows the **split-apply-combine** pattern: split the data into groups based on a key, apply a function to each group independently, then combine the results back into a single structure.

Grouping and aggregating

```
df.groupby('city')['age'].mean()

df.groupby('city').agg(
    avg_age=('age', 'mean'),
    count=('age', 'size')
)

df.groupby('city').agg({'age': ['mean', 'max', 'min']})
```

Output:

```
city
Aswan          23.0
Cairo          25.0
Giza           21.0
Alexandria     22.0
Name: age, dtype: float64
```

11. Merging, Joining, and Concatenating

11.1 merge — SQL-style joins

Merging two DataFrames

```
orders = pd.DataFrame({'order_id': [1, 2, 3], 'customer_id': [10, 20, 10]})
customers = pd.DataFrame({'customer_id': [10, 20], 'name': ['Nour', 'Ali']})

pd.merge(orders, customers, on='customer_id', how='left')
```

Output:

```
   order_id  customer_id  name
0         1           10  Nour
1         2           20   Ali
2         3           10  Nour
```

Note: how can be 'inner' (default), 'left', 'right', or 'outer', matching SQL join semantics.

11.2 concat — stacking DataFrames

Concatenating DataFrames

```
df_a = pd.DataFrame({'x': [1, 2]})
df_b = pd.DataFrame({'x': [3, 4]})
pd.concat([df_a, df_b], ignore_index=True)   # stack rows
pd.concat([df_a, df_b], axis=1)              # stack columns side by side
```

12. Pivot Tables and Cross-tabulation

pivot_table for summarizing data

```
sales = pd.DataFrame({
    'region': ['East', 'East', 'West', 'West'],
    'product': ['A', 'B', 'A', 'B'],
    'revenue': [100, 150, 200, 90]
})
sales.pivot_table(values='revenue', index='region', columns='product', aggfunc='sum')
```

Output:

```
product   A    B
region
East    100  150
West    200   90
```

13. apply, map, and Lambda Functions

Transforming data with apply / map

```
df['age_group'] = df['age'].apply(lambda x: 'adult' if x >= 22 else 'young')

# map: element-wise substitution on a Series using a dict or function
df['city_short'] = df['city'].map({'Cairo': 'CAI', 'Giza': 'GIZ'})

# apply on a DataFrame runs across rows or columns
df[['age']].apply(lambda col: col.max() - col.min())
```

14. String Methods (.str accessor)

Vectorized string operations

```
df['name'].str.upper()
df['name'].str.lower()
df['name'].str.contains('a', case=False)
df['name'].str.replace('o', '0')
df['name'].str.len()
df['city'].str.split(' ').str[0]   # first word of each entry
```

15. Working with Dates and Time Series

Parsing and using datetime columns

```
df['date'] = pd.to_datetime(['2026-01-01', '2026-02-15', '2026-03-20'])
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['weekday'] = df['date'].dt.day_name()

ts = df.set_index('date')
ts.resample('M').mean(numeric_only=True)   # monthly resampling
```

16. Duplicates, Unique Values, and Value Counts

Working with duplicates and category counts

```
df.duplicated()              # boolean mask of duplicate rows
df.drop_duplicates()         # remove duplicate rows
df['city'].unique()          # array of distinct values
df['city'].nunique()         # number of distinct values
df['city'].value_counts()    # frequency of each value, sorted descending
```

17. Mini Case Study — Putting It Together

A short end-to-end example combining several pandas operations on a small sales dataset.

End-to-end mini workflow

```
import pandas as pd

df = pd.DataFrame({
    'product': ['Laptop', 'Phone', 'Laptop', 'Tablet', 'Phone'],
    'region':  ['East', 'East', 'West', 'West', 'East'],
    'units':   [5, 10, 3, None, 7],
    'price':   [1000, 500, 1000, 300, 500]
})

# 1. Clean missing data
df['units'] = df['units'].fillna(df['units'].median())

# 2. Feature engineering
df['revenue'] = df['units'] * df['price']

# 3. Aggregate by product
summary = df.groupby('product').agg(
    total_units=('units', 'sum'),
    total_revenue=('revenue', 'sum')
).sort_values('total_revenue', ascending=False)

print(summary)
```

Output:

```
         total_units  total_revenue
product
Laptop           8.0         8000.0
Phone           17.0         8500.0
Tablet           5.0         1500.0
```

18. Quick Reference Cheat Sheet

Task	Code
Create DataFrame	pd.DataFrame(dict_or_list)
Read CSV	pd.read_csv('file.csv')
First/last rows	df.head(n) / df.tail(n)
Shape / dtypes	df.shape / df.dtypes
Select column(s)	df['col'] / df[['a','b']]
Label-based selection	df.loc[row, col]
Position-based selection	df.iloc[row, col]
Filter rows	df[df['col'] > x]
Sort	df.sort_values('col')
Missing values	df.isna(), df.dropna(), df.fillna(v)
New column	df['new'] = expression
Rename columns	df.rename(columns={'old':'new'})
Drop rows/cols	df.drop(columns=[...]) / df.drop(index=[...])
Group and aggregate	df.groupby('key').agg(...)
Join tables	pd.merge(a, b, on='key', how='left')
Stack DataFrames	pd.concat([a, b])
Pivot summary	df.pivot_table(values, index, columns, aggfunc)
Apply function	df['col'].apply(func)
String ops	df['col'].str.method()
Parse dates	pd.to_datetime(df['col'])
Unique / counts	df['col'].unique() / .value_counts()
Drop duplicates	df.drop_duplicates()
Write CSV	df.to_csv('out.csv', index=False)

Learning tip: pandas methods almost always return a *new* DataFrame or Series rather than modifying in place (unless inplace=True is passed). Chain methods together for concise pipelines, e.g. `df[df.age > 20].groupby('city').mean().sort_values('age')`.